Enterprise Knowledge Assistant

---

1. Project Overview

1.1 Business Problem

Modern organizations store critical information across multiple disconnected systems such as:

- Knowledge repositories
- Policy documents
- CRM systems
- Ticketing tools
- Project management platforms

Employees face the following challenges:

- Time-consuming information retrieval
- Lack of centralized knowledge access
- Inconsistent and outdated responses
- Reduced productivity

1.2 Solution Approach

This project implements an **AI-powered Enterprise Knowledge Assistant** using **Agentic AI architecture**.

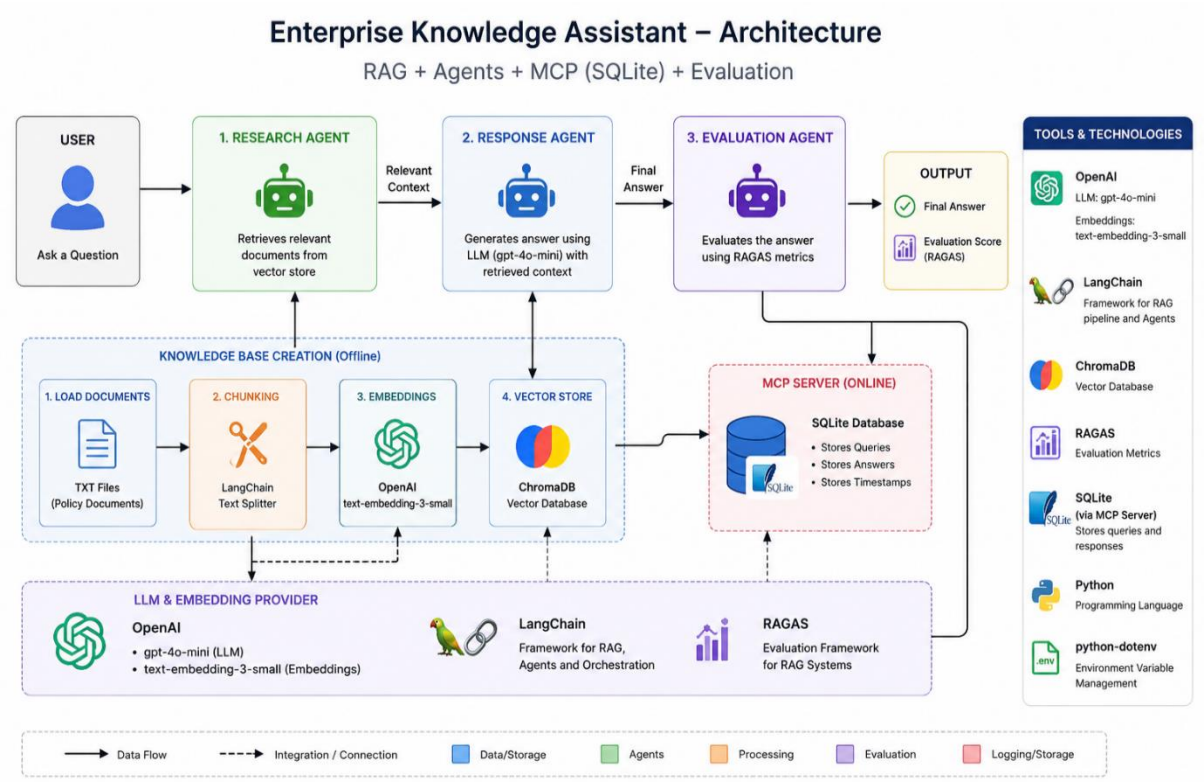
Key capabilities:

- Retrieve enterprise knowledge using **RAG (Retrieval-Augmented Generation)**
- Generate context-aware responses using **LLMs**
- Evaluate response quality using **RAGAS metrics**
- Interact with external systems via **MCP Server**
- Coordinate multiple agents using **CrewAI**

---

2. Architecture

2.1 High-Level Architecture



Components:

- User Interface / CLI
- CrewAI Multi-Agent System
- RAG Pipeline
- Vector Database
- MCP Server Integration
- RAGAS Evaluation Layer

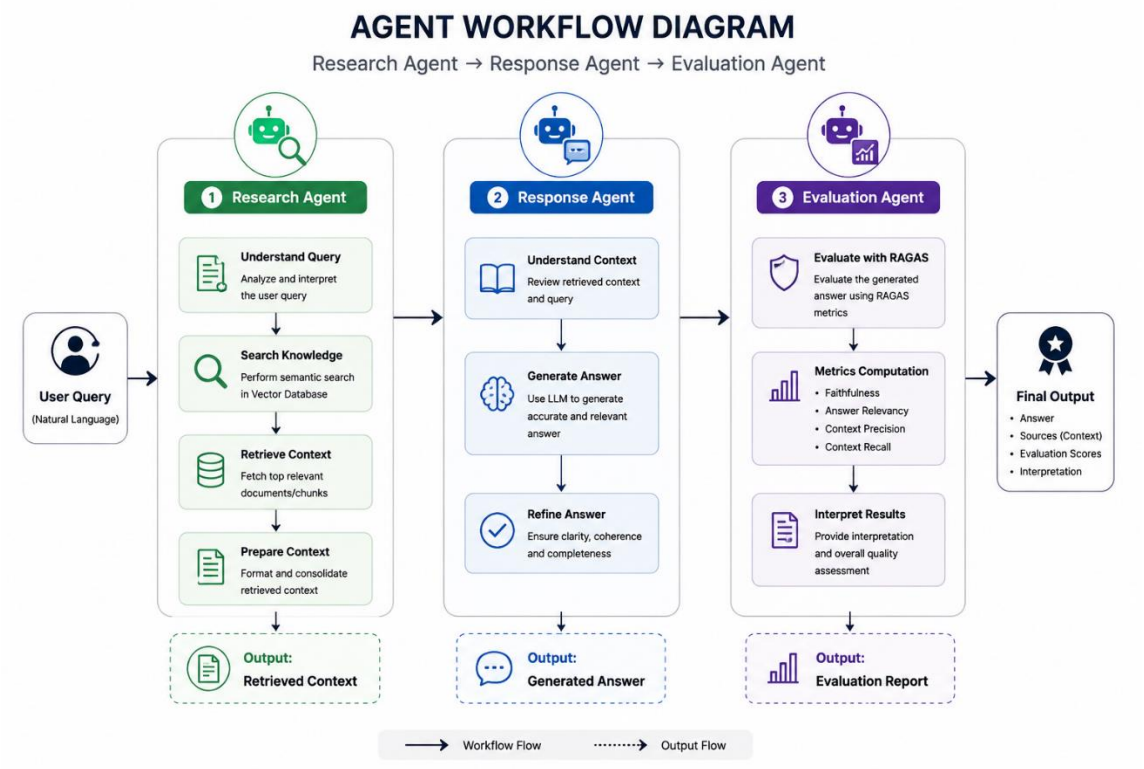
Flow:

1. User query received
2. Research Agent retrieves context (RAG)
3. Response Agent generates answer
4. MCP tools are used if required
5. Evaluation Agent evaluates output using RAGAS
6. Final response + evaluation shown

2.2 Agent Workflow Diagram

Workflow:

1. **Research Agent**
   → Retrieves relevant context from vector DB
2. **Response Generation Agent**
   → Generates answer using retrieved context
3. **Evaluation Agent**
   → Evaluates output using RAGAS metrics



---

3. Setup Instructions

3.1 Environment Setup

Requirements:

- Python 3.10+
- Virtual Environment

python -m venv venv

venv\Scripts\activate # Windows

source venv/bin/activate # Mac/Linux

3.2 Dependency Installation

pip install -r requirements.txt

3.3 Configuration Steps

Create .env file:

OPENAI_API_KEY=your_api_key

---

4. Execution Steps

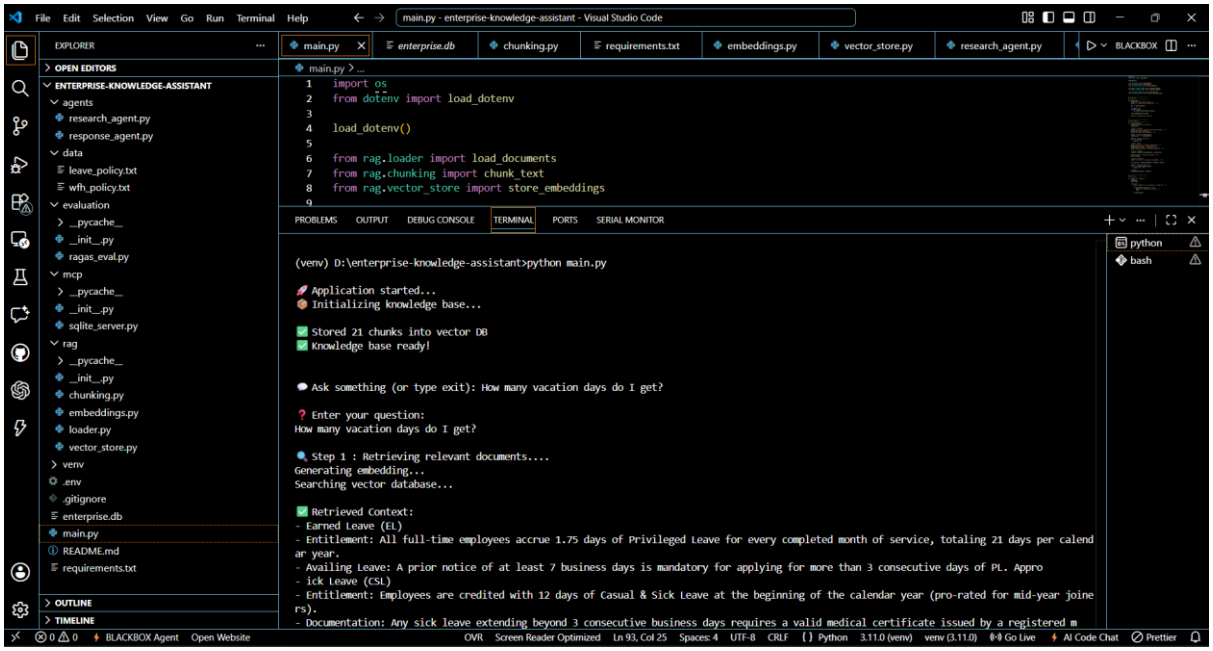
4.1 Run Application

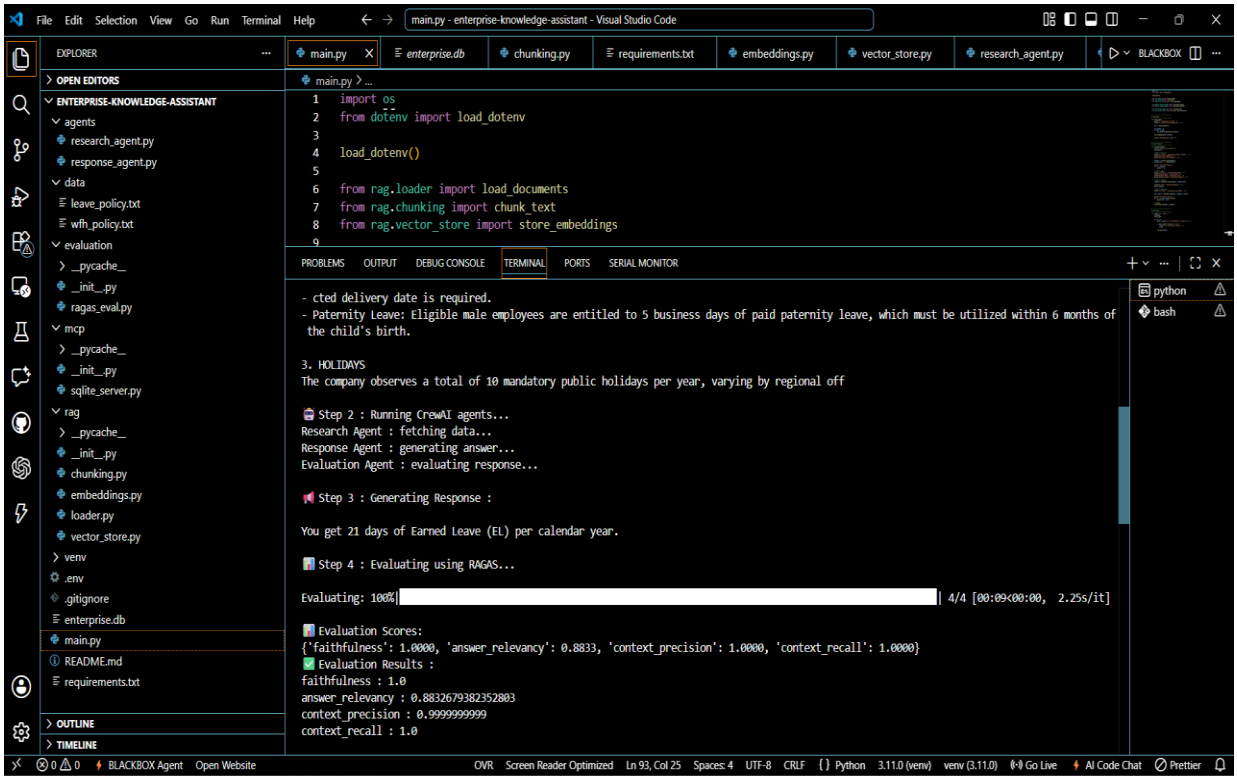
python main.py

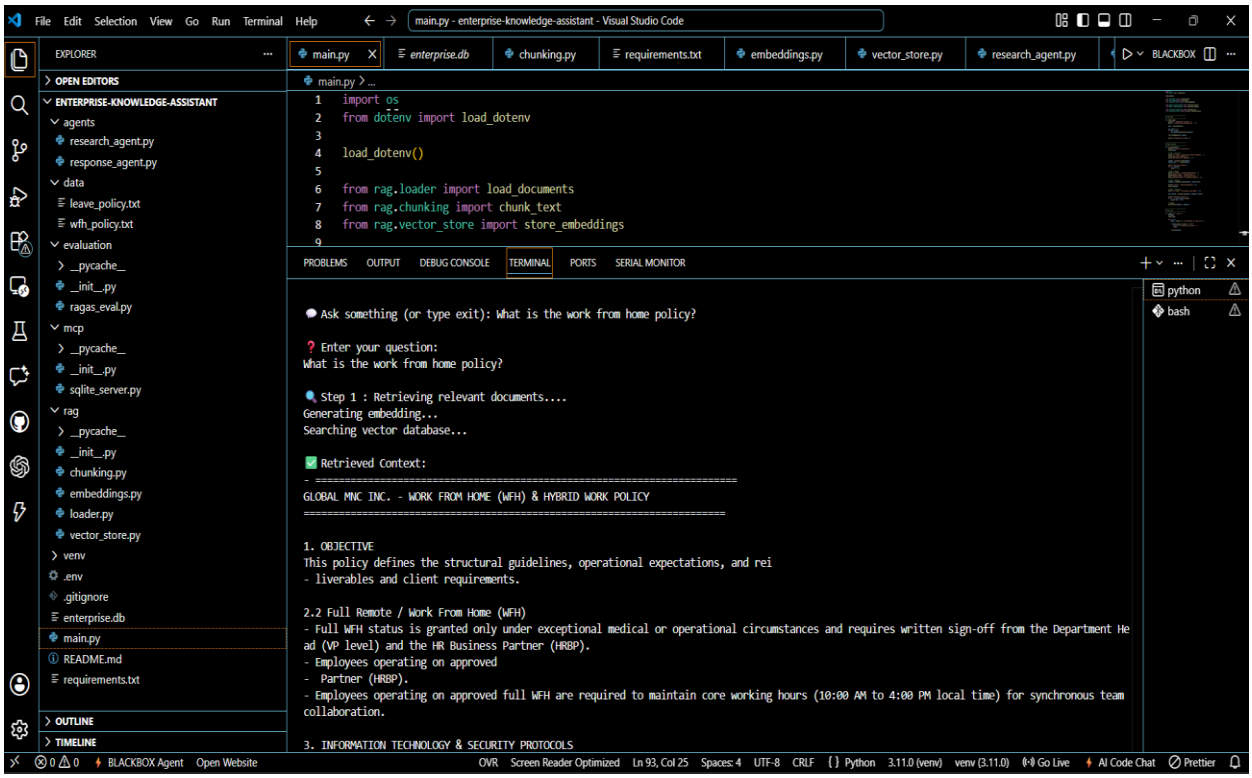
4.2 Sample Inputs

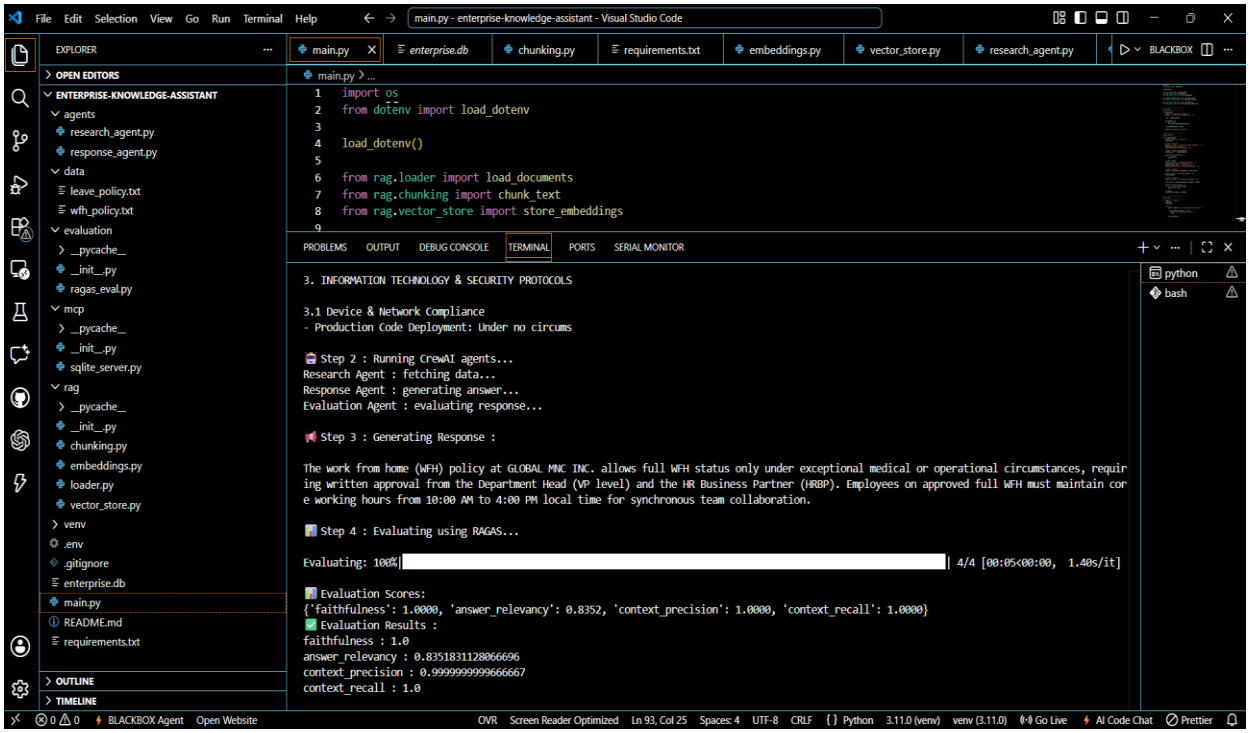
- "What is the company leave policy?"
- "How many vacation days do I get??"
- "What is work-from-home policy in our company?"

4.3 Outputs




File Edit Selection View Go Run Terminal Help
main.py - enterprise-knowledge-assistant - Visual Studio Code
EXPLORER
OPEN EDITORS
ENTERPRISE-KNOWLEDGE-ASSISTANT
agents
research_agent.py
response_agent.py
data
leave_policy.txt
wfh_policy.txt
evaluation
__pycache__
__init__.py
ragas_eval.py
mcp
__pycache__
__init__.py
sqlite_server.py
rag
__pycache__
__init__.py
chunking.py
embeddings.py
loader.py
vector_store.py
venv
.env
.gitignore
enterprise.db
main.py
README.md
requirements.txt
OUTLINE
TIMELINE
main.py
enterprise.db
chunking.py
requirements.txt
embeddings.py
vector_store.py
research_agent.py
BLACKBOX
main.py > ...
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()
5
6 from rag.loader import load_documents
7 from rag.chunking import chunk_text
8 from rag.vector_store import store_embeddings
9
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
python
bash
- cted delivery date is required.
- Paternity Leave: Eligible male employees are entitled to 5 business days of paid paternity leave, which must be utilized within 6 months of the child's birth.
3. HOLIDAYS
The company observes a total of 10 mandatory public holidays per year, varying by regional off
Step 2 : Running CrewAI agents...
Research Agent : fetching data...
Response Agent : generating answer...
Evaluation Agent : evaluating response...
Step 3 : Generating Response :
You get 21 days of Earned Leave (EL) per calendar year.
Step 4 : Evaluating using RAGAS...
Evaluating: 100%| 4/4 [00:09<00:00, 2.25s/it]
Evaluation Scores:
{'faithfulness': 1.0000, 'answer_relevancy': 0.8833, 'context_precision': 1.0000, 'context_recall': 1.0000}
Evaluation Results :
faithfulness : 1.0
answer_relevancy : 0.8832679382352803
context_precision : 0.9999999999
context_recall : 1.0
0 0 BLACKBOX Agent Open Website
OVR Screen Reader Optimized Ln 93, Col 25 Spaces: 4 UTF-8 CRLF { } Python 3.11.0 (venv) venv (3.11.0) Go Live AI Code Chat Prettier


File Edit Selection View Go Run Terminal Help
main.py - enterprise-knowledge-assistant - Visual Studio Code
EXPLORER
OPEN EDITORS
ENTERPRISE-KNOWLEDGE-ASSISTANT
agents
research_agent.py
response_agent.py
data
leave_policy.txt
wfh_policy.txt
evaluation
__pycache__
__init__.py
ragas_eval.py
mcp
__pycache__
__init__.py
sqlite_server.py
rag
__pycache__
__init__.py
chunking.py
embeddings.py
loader.py
vector_store.py
venv
.env
.gitignore
enterprise.db
main.py
README.md
requirements.txt
OUTLINE
TIMELINE
main.py
enterprise.db
chunking.py
requirements.txt
embeddings.py
vector_store.py
research_agent.py
BLACKBOX
main.py > ...
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()
5
6 from rag.loader import load_documents
7 from rag.chunking import chunk_text
8 from rag.vector_store import store_embeddings
9
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
python
bash
Ask something (or type exit): What is the work from home policy?
Enter your question:
What is the work from home policy?
Step 1 : Retrieving relevant documents....
Generating embedding...
Searching vector database...
Retrieved Context:
- ==========
GLOBAL MNC INC. - WORK FROM HOME (WFH) & HYBRID WORK POLICY
==========
1. OBJECTIVE
This policy defines the structural guidelines, operational expectations, and rei
- liverables and client requirements.
2.2 Full Remote / Work From Home (WFH)
- Full WFH status is granted only under exceptional medical or operational circumstances and requires written sign-off from the Department He ad (VP level) and the HR Business Partner (HRBP).
- Employees operating on approved
- Partner (HRBP).
- Employees operating on approved full WFH are required to maintain core working hours (10:00 AM to 4:00 PM local time) for synchronous team collaboration.
3. INFORMATION TECHNOLOGY & SECURITY PROTOCOLS
0 0 BLACKBOX Agent Open Website
OVR Screen Reader Optimized Ln 93, Col 25 Spaces: 4 UTF-8 CRLF { } Python 3.11.0 (venv) venv (3.11.0) Go Live AI Code Chat Prettier



---

5. RAG Design

5.1 Document Source

- Internal policy PDFs
- Knowledge base documents
- Text/CSV files

5.2 Chunking Strategy

- Chunk size: 500–1000 tokens
- Overlap: 100–200 tokens
- Method: Recursive text splitter

5.3 Embedding Model

- OpenAI Embeddings (text-embedding-3-small)

5.4 Vector Database

- ChromaDB

Features:

- Fast similarity search
- Persistent storage
- Scalable retrieval

---

6. CrewAI Design

6.1 Agents Created

1. **Research Agent**
2. **Response Generation Agent**
3. **Evaluation Agent**

6.2 Agent Responsibilities

Research Agent

- Retrieves relevant documents
- Performs semantic search

Response Agent

- Uses LLM to generate answer
- Uses retrieved context

Evaluation Agent

- Uses RAGAS for evaluation
- Computes quality metrics

6.3 Task Flow

1. User query received
2. Research Agent → retrieves context
3. Response Agent → generates answer

4. Evaluation Agent → evaluates answer
5. Output displayed

---

7. MCP Integration

7.1 MCP Server Used

- SQLite MCP Server

7.2 Tools Exposed

- Query database
- Fetch structured records
- Read files

7.3 Use Case Implemented

Example:

- Fetch employee data from SQLite
- Retrieve logs from filesystem
- Enhance response using external data

---

8. RAGAS Evaluation

8.1 Metrics Collected

- **Faithfulness**
- **Answer Relevancy**
- **Context Precision**
- **Context Recall**

8.2 Results Obtained (Example)

Metric	Score
Faithfulness	0.90
Answer Relevancy	0.91
Context Precision	0.85
Context Recall	0.82

8.3 Analysis of Results

- High faithfulness → Answer aligns with context
- High relevancy → Query properly addressed
- Precision moderate → Some irrelevant chunks retrieved
- Recall slightly lower → Some useful context missing

Conclusion:
System performs well but can improve retrieval optimization.

---

9. Challenges and Learnings

9.1 Key Implementation Challenges

- Managing multi-agent coordination
- Designing efficient chunking strategy
- Integrating MCP tools with agents
- Handling hallucination in LLM outputs
- Evaluating responses using RAGAS

9.2 Lessons Learned

- Agentic workflows improve modularity
- RAG significantly improves factual accuracy
- Evaluation (RAGAS) is critical for production systems

- MCP enables real-world system integration
- Prompt engineering plays a crucial role

---

10. Conclusion

The project successfully demonstrates:

- Multi-agent orchestration using CrewAI
- End-to-end RAG pipeline
- Quality evaluation using RAGAS
- External system integration via MCP

This solution can be extended into a **production-grade enterprise assistant**.
